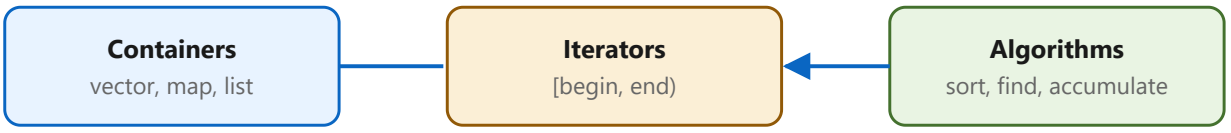
The C++ Standard Template Library

Containers hold data, algorithms work on it, iterators connect them. Container choice, operation costs and iterator invalidation are the parts that matter most.

The big picture



Algorithms never touch containers, only iterator ranges.

One sort works on any container that offers the right iterators.

PART	WHAT IT IS	EXAMPLES
Containers	classes that own and organise a collection of elements	`vector`, `map`, `set`, `deque`
Iterators	generalised pointers that walk a container	`v.begin()`, `m.find(k)`
Algorithms	generic functions that operate on iterator *ranges*	`sort`, `find`, `accumulate`

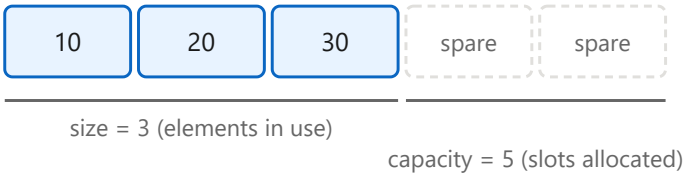
Algorithms see only `[first, last)`, so one `std::sort` serves a vector, array or deque. That separation is the design.

```
#include <vector>       // std::vector
#include <map>          // std::map, std::multimap
#include <unordered_map>// std::unordered_map
#include <set>          // std::set
#include <algorithm>    // std::sort, std::find, std::transform ...
#include <numeric>      // std::accumulate, std::iota
#include <string>       // std::string
```

DEFAULT CHOICE `std::vector`. Contiguous memory is so cache-friendly it often beats "better" containers.

`vector`: the workhorse

buffer on the heap



push_back, buffer full:
allocate bigger, move all, free old
= amortised O(1), but every iterator and pointer into it is invalidated

reserve() grows capacity only; resize() changes size.

A `vector` is one contiguous heap array with a **size** and a **capacity**. When full, `push_back` allocates a bigger buffer, moves everything and frees the old one: occasional O(n), **amortised O(1)**.

STEP THROUGH **vector growth and a dangling pointer**

```
1  std::vector<int> v;
2  v.reserve(2);
3  v.push_back(10);
4  v.push_back(20);
5  int* p = &v[0];
6  v.push_back(30);
7  // p now?
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `std::vector<int> v;`	An empty vector owns no heap memory yet.	**`v = size 0, cap 0`**
2	line 2 `v.reserve(2);`	**reserve(2)** allocates room for 2 ints but creates no elements: capacity 2, size 0.	**`v = size 0, cap 2 | buffer = _ , _ (heap)`**
3	line 3 `v.push_back(10);`	Fits in the spare capacity: no reallocation.	**`v = size 1, cap 2 | buffer = 10 , _ (heap)`**
4	line 4 `v.push_back(20);`	The buffer is now full (size == capacity).	**`v = size 2, cap 2 | buffer = 10 , 20 (heap)`**
5	line 5 `int* p = &v[0];`	We take the address of the first element. This pointer is only valid while the buffer stays where it is.	`v = size 2, cap 2 |` **`p = 0x5000`** `| buffer = 10 , 20 (heap)`
6	line 6 `v.push_back(30);`	No room: the vector allocates a **bigger buffer at a new address**, moves the elements over and frees the old one. (The growth factor is implementation-defined; 2x is common.)	**`v = size 3, cap 4`** `| p = 0x5000` ~~`old buffer = 10 , 20 (heap)`~~ **`| new buffer = 10 , 20 , 30 , _ (heap)`**
7	line 7 `// p now?`	**p dangles.** The same applies to every iterator and reference into the vector. Fix: `reserve` enough up front, or do not keep pointers across a growing operation.	`v = size 3, cap 4 | p = 0x5000` ~~`old buffer = 10 , 20 (heap)`~~ `| new buffer = 10 , 20 , 30 , _ (heap)`

```cpp
std::vector<int> v;
v.reserve(100);          // capacity 100, size still 0: no reallocation for the next 100 push_backs
v.push_back(1);          // copy/move an existing object in
v.emplace_back(2);       // construct the element in place: no temporary
v.resize(10);            // size becomes 10; new elements are value-initialised (0)

v[3];                    // no bounds check: out-of-range is undefined behavior
v.at(3);                 // bounds-checked: throws std::out_of_range
v.front(); v.back(); v.data();   // data() is the raw int* for C APIs
v.size(); v.capacity(); v.empty();
```

RESERVE VS RESIZE `reserve(n)` only allocates; `resize(n)` creates elements. `v[5] = x` after only `reserve(10)` is UB.

BRACES VS PARENTHESES

```cpp
std::vector<int> a(3, 7);   // parentheses: size 3, every element 7   -> {7, 7, 7}
std::vector<int> b{3, 7};   // braces: the initializer_list constructor wins  ->  {3, 7}
```

With braces, a constructor taking `std::initializer_list` is preferred whenever it can match.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 3.3 Library vector type (PDF pages 125 to 134)
- 9.3 Sequential container operations (PDF pages 370 to 383)
- 9.4 How a vector grows (PDF pages 384 to 388)

Programming: Principles and Practice (Stroustrup)

- 3.5 to 3.6 Functions; vector introduced (PDF pages 81 to 89)
- 17.6 to 17.9 Other operations, remaining Vector problems, changing size (PDF pages 513 to 523)

Effective Modern C++ (Meyers)

- Item 42, Consider emplacement instead of insertion (PDF pages 310 to 320)

C++ Move Semantics (Josuttis)

- Chapter 15 Move semantics in standard library types (PDF pages 229 to 250)

Iterators

An iterator acts like a pointer: `*it`, `++it`, `it == end`. `end()` is **one past the last**; ranges are half-open `[begin, end)`.

```
std::vector<int> v{1, 2, 3};
for (auto it = v.begin(); it ≠ v.end(); ++it) std::cout << *it;   // use ≠ not <, it works for every container
for (int x : v) std::cout << x;           // range-for is exactly this loop in disguise
for (auto it = v.rbegin(); it ≠ v.rend(); ++it) { }   // reverse iterators
for (auto it = v.cbegin(); it ≠ v.cend(); ++it) { }   // cbegin/cend: read-only, *it cannot be modified
auto it2 = std::next(v.begin(), 2);       // advance without assuming random access
```

CATEGORY	CAN DO	CONTAINERS

Forward	`++`, read (multi-pass)	`forward_list`, unordered containers
Bidirectional	also `--`	`list`, `map`, `set`
Random access	also `it + n`, `it[i]`, `it1 - it2`, `<`	`vector`, `deque`, `array`, `string`

So `std::sort` needs random access and fails on `std::list` (use `list.sort()`).

Iterator adaptors: filling a container

An algorithm like `std::copy` **assigns** through its output iterator, so the destination must already be big enough. An **inserter** turns each assignment into an insertion instead:

ADAPTOR	EACH ASSIGNMENT DOES
`std::back_inserter(c)`	`c.push_back(x)`
`std::front_inserter(c)`	`c.push_front(x)` (so the order reverses; needs `deque` / `list`)
`std::inserter(c, pos)`	`c.insert(pos, x)` (works for `set` and `map`)

```
std::vector<int> src{3, 1, 2}, out;
std::copy(src.begin(), src.end(), std::back_inserter(out));      // out is
{3, 1, 2}: it grew
// std::copy(src.begin(), src.end(), out.begin());               // BUG if
out is empty: writes past the end

std::set<int> sorted;
std::copy(src.begin(), src.end(), std::inserter(sorted, sorted.begin()));
```

`std::reverse_iterator` (what `rbegin()` returns) and `std::move_iterator` (moves instead of copying) are the other two adaptors worth knowing.

Iterator invalidation

Changing a container can **invalidate** iterators, pointers and references; using one is UB.

CONTAINER	OPERATION	WHAT IS INVALIDATED

`vector`	`push_back`, `insert`	**all** if it reallocated; otherwise those at or after the insertion point
`vector`	`erase`	the erased element and everything after it
`deque`	insert/erase in the middle	everything; push at either end invalidates iterators but not references
`list`, `map`, `set`	insert	nothing
`list`, `map`, `set`	erase	only the erased element
`unordered_*`	insert that triggers a rehash	all iterators (references stay valid)

▼ SPOT THE BUG Erasing inside a loop

```
std::vector<int> v{1, 2, 3, 4};
for (auto it = v.begin(); it ≠ v.end(); ++it)
    if (*it % 2 == 0) v.erase(it);          // BUG
```

`erase` invalidates `it`, and the loop then does `++it` on a dead iterator. The correct idiom uses the iterator `erase` returns:

```
for (auto it = v.begin(); it ≠ v.end(); ) {
    if (*it % 2 == 0) it = v.erase(it);     // erase returns the next
valid iterator
    else ++it;
}
// or, in C++20:
std::erase_if(v, [](int x) { return x % 2 == 0; });
```

Step through a correct erase loop:

STEP THROUGH **Erasing inside a loop: use the iterator erase returns**

```
1  std::vector<int> v{1, 2, 3, 4};
2  for (auto it = v.begin(); it != v.end(); ) {
3      if (*it % 2 == 0) it = v.erase(it);
4      else ++it;
5  }
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `std::vector<int> v{1, 2, 3, 4};`	The vector holds 1, 2, 3, 4. We want to remove the even numbers.	**v[0] = 1** | **v[1] = 2** | **v[2] = 3** | **v[3] = 4**
2	line 2 `for (auto it = v.begin(); it ≠ v.end(); ) {`	**it** starts at the first element.	v[0] = 1 | v[1] = 2 | v[2] = 3 | v[3] = 4 | **it = -> v[0]**
3	line 5 `}`	1 is odd, so `++it`. Now it points at 2.	v[0] = 1 | v[1] = 2 | v[2] = 3 | v[3] = 4 | **it = -> v[1]**
4	line 3 `if (*it % 2 == 0) it = v.erase(it);`	2 is even: `v.erase(it)` shifts the later elements down and **returns an iterator to the next element** (now 3). The old `it` is invalid, so we must overwrite it with the return value.	v[0] = 1 | **v[1] = 3** | **v[2] = 4** | ~~v[3] = gone~~ | it = -> v[1]
5	line 5 `}`	3 is odd: `++it`. Note that after an erase we did **not** advance: the next element had already moved into that slot.	v[0] = 1 | v[1] = 3 | v[2] = 4 | **it = -> v[2]**
6	line 3 `if (*it % 2 == 0) it = v.erase(it);`	4 is even: erase it. The returned iterator is `end()`, so the loop stops. Result: **1, 3**.	v[0] = 1 | v[1] = 3 ~~| v[2] = gone~~ | **it = end()**
7	line 3 `if (*it % 2 == 0) it = v.erase(it);`	The buggy version `v.erase(it)` followed by `++it` uses a dead iterator (undefined behavior) and also skips an element. C++20 shortcut: `std::erase_if(v, pred)`.	v[0] = 1 | v[1] = 3 | it = end()

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 3.4 Introducing iterators (PDF pages 135 to 141)
- 10.4 Revisiting iterators (PDF pages 430 to 438)
- 10.5 Structure of generic algorithms (PDF pages 439 to 443)

Programming: Principles and Practice (Stroustrup)

- 19.1 to 19.3, 19.6 Containers, sequences and iterators, linked lists, vector/list/string (PDF pages 559 to 572 and 585 to 586)

Other sequence containers

CONTAINER	STORAGE	BEST AT	WATCH OUT
`std::array<T,N>`	fixed size, inline (stack)	size known at compile time; zero overhead	cannot grow
`std::vector<T>`	one contiguous heap block	almost everything; fast iteration and indexing	insert/erase in the middle is O(n)
`std::deque<T>`	chunks of arrays	fast push/pop at **both** ends, still indexable	not contiguous
`std::list<T>`	doubly linked nodes	O(1) insert/erase anywhere *if you hold an iterator*; iterators stay valid	no indexing, a heap allocation per element, poor cache behaviour
`std::forward_list<T>`	singly linked nodes	minimal memory list	forward iteration only

OPERATION	VECTOR	DEQUE	LIST
index `[i]`	O(1)	O(1)	not available
`push_back`	amortised O(1)	O(1)	O(1)

`push_front`	O(n)	O(1)	O(1)
insert / erase in the middle	O(n)	O(n)	O(1) at a known position
find a value	O(n)	O(n)	O(n)

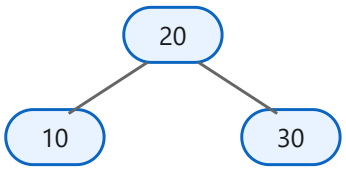
READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 9.1 to 9.2 Overview and the container library (PDF pages 355 to 369)
- 9.6 Container adaptors (PDF pages 397 to 402)
- 10.6 Container-specific algorithms (PDF pages 444 to 447)

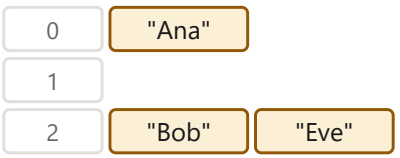
Associative and unordered containers

map / set: balanced tree, sorted, O(log n)



in-order walk = sorted order

unordered_map: hash table, average O(1)



hash(key) picks a bucket; collisions chain (Bob, Eve)

Need order or range queries: map. Plain lookup: unordered_map.

	`map` / `set`	`unordered_map` / `unordered_set`
Implementation	balanced binary tree (usually red-black)	hash table
Order of iteration	**sorted** by key	unspecified
find / insert / erase	O(log n)	average O(1), worst case O(n)
Key requirement	`operator<` (a strict weak ordering)	a hash function and `operator==`
Range queries (`lower_bound`)	yes	no

```cpp
std::map<std::string, int> age;
age["Ana"] = 30;                          // operator[] INSERTS a default (0) if the key is missing
age.insert({"Bob", 25});                  // does nothing if "Bob" already exists
age.insert_or_assign("Bob", 26);          // C++17: insert or overwrite

if (auto it = age.find("Ana"); it ≠ age.end())   // find never inserts
    std::cout << it→second;                       // it→first is the key, it→second the value

for (const auto& [name, years] : age)     // structured bindings; sorted by name
    std::cout << name << ' ' << years << '\n';

std::unordered_set<int> seen;
if (!seen.insert(5).second) { /* second == false means it was already there */ }
```

▼ PREDICT **Does looking up make the map bigger?**

```cpp
std::map<std::string, int> m;
if (m["x"] == 0) { }
std::cout << m.size();
```

```
1
```

`operator[]` on a missing key **inserts** it with a default-constructed value. To only check, use `m.find("x")`, `m.count("x")` or `m.contains("x")` (C++20).

`try_emplace`, `multimap` and `multiset`

CALL	IF THE KEY ALREADY EXISTS
`m[k] = v`	overwrites (and inserts a default first if missing)
`m.insert({k, v})` / `m.emplace(k, v)`	does nothing, but **may already have moved from or built the value**

`m.try_emplace(k, args...)` (C++17)	does nothing and **guarantees the arguments are not touched**: compiled, a moved-in 40-character string was intact afterwards
`m.insert_or_assign(k, v)` (C++17)	overwrites

```
std::map<int, std::string> a{{1, "old"}};
std::string payload(40, 'p');
a.try_emplace(1, std::move(payload));     // key 1 exists: nothing happens, payload still holds 40 chars
a.try_emplace(2, std::move(payload));     // new key: the string is moved into the map

std::multimap<std::string, int> mm{{"fruit", 1}, {"fruit", 2}, {"veg", 9}};
// duplicate keys allowed
auto [lo, hi] = mm.equal_range("fruit");   // the range of all entries with that key
mm.count("fruit");                         // 2: there is no operator[] because a key may have many values
```

- `multimap` and `multiset` allow duplicate keys and keep equal keys in insertion order. There is no `operator[]`.
- `try_emplace` is the right call for "insert if absent" when building the value is expensive or the argument is move-only.

Container adaptors

They wrap another container and restrict the interface: `std::stack` (LIFO: `push/pop/top`), `std::queue` (FIFO: `push/pop/front`) and `std::priority_queue` (a heap: `top()` is the **largest** element by default).

```
std::priority_queue<int> maxHeap;                                  // largest first
std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;   // smallest first
```

Step through `operator[]`:

STEP THROUGH **std::map::operator[] inserts, find does not**

```
1  std::map<std::string, int> m;
2  if (m["x"] == 0) { }
3  m.contains("y");
4  std::cout << m.size();
```

STEP	RUNS	WHAT HAPPENS	MEMORY AFTER (CHANGED IN BOLD)
1	line 1 `std::map<std::string, int> m;`	An empty map.	**`m = size 0 (empty)`**
2	line 2 `if (m["x"] == 0) { }`	`m["x"]` on a missing key **inserts** a node with a default-constructed value (0) and returns a reference to it. The comparison then sees 0.	**`m = size 1 | node "x" = "x" -> 0 (heap)`**
3	line 3 `m.contains("y");`	`contains("y")` (or `find`, `count`) only **looks**: nothing is inserted.	`m = size 1 | node "x" = "x" -> 0 (heap)`
4	line 4 `std::cout << m.size();`	Prints **1**. A lookup that grew the map: use `find` / `contains` to test, `at()` to read-or-throw. prints: `1\n`	`m = size 1 | node "x" = "x" -> 0 (heap)`

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 11.1 to 11.2 Using an associative container, overview (PDF pages 449 to 456)
- 11.3 Operations on associative containers (PDF pages 457 to 471)

Programming: Principles and Practice (Stroustrup)

- 20.1 to 20.6 map, unordered_map, set, container overview (PDF pages 591 to 609)

Custom keys: comparators and hashes

A `map` / `set` needs an **ordering**; an `unordered_map` / `unordered_set` needs a **hash and equality**. For your own key types, you provide them.

```
struct NoCase {                                          // an ordering that
ignores case
    bool operator()(const std::string& a, const std::string& b) const {
        return std::lexicographical_compare(a.begin(), a.end(), b.begin(),
b.end(),
            [](unsigned char x, unsigned char y) { return std::tolower(x) <
std::tolower(y); });
    }
};
std::map<std::string, int, NoCase> m;                    // the comparator is
part of the TYPE
m["Alpha"] = 1; m["ALPHA"] = 2;                          // the same key:
size is 1 (compiled: 2 keys with "beta")

struct Product { std::string shop; int id; bool operator==(const Product&)
const = default; };
struct ProductHash {
    std::size_t operator()(const Product& p) const {
        std::size_t h = std::hash<std::string>{}(p.shop);
        h ^= std::hash<int>{}(p.id) + 0x9e3779b97f4a7c15ULL + (h << 6) + (h
>> 2);   // combine the member hashes
        return h;
    }
};
std::unordered_map<Product, double, ProductHash> price;
price[{"north", 7}] = 101.5;
```

CONTAINER	KEY MUST PROVIDE	RULE
`map`, `set`	a **strict weak ordering** (`operator<` or a comparator)	irreflexive and transitive; equivalence means "neither is less". `≤` is **not** valid: undefined behaviour
`unordered_map`, `unordered_set`	a hash and `operator==`	equal keys **must** have equal hashes; a poor hash makes every lookup O(n)

- **Transparent comparators:** `std::map<std::string, int, std::less<>>` lets `find` take a `string_view` or `const char*` without building a temporary `std::string` (compiled: it found `"apple"` from a `string_view`). C++20 adds the same for unordered containers.
- **Never change a key** while it is in the container: its position or bucket would be wrong. Keys are `const` for that reason.
- **Hash quality:** combine *all* fields; avoid returning just one field. `std::hash` for `int` is the identity in libstdc++, which is fine.
- A `rehash` (growth beyond `max_load_factor`) invalidates iterators; `reserve(n)` up front avoids it, and avoids the allocations on a hot path.
- Ordered or hashed? Ordered when you need sorted iteration or range queries (`lower_bound`); hashed for plain lookup. For a few dozen keys a sorted `vector` and binary search often beats both.

KEY POINT "How do you use your own type as a key in `unordered_map`?" Provide `operator==` and a hash functor (or specialise `std::hash`), combining all members, and make sure equal objects hash equally.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 11.3 Operations on associative containers (PDF pages 457 to 471)
- 11.4 The unordered containers (PDF pages 472 to 477)

Which container should I use?

Pick what you mostly do:

ADVISOR **Which container should I use?**

I need to: Iterate over items, access by index, mostly append at the end

Use `std::vector`.
Contiguous memory: fastest iteration and indexing, amortised O(1) push_back. This is the default choice.

KEY POINT "vector vs list?" Answer with *memory*: contiguous means cache-friendly, so vector wins even for middle inserts at modest sizes. List wins for stable iterators and cheap splicing.

READ MORE

Programming: Principles and Practice (Stroustrup)

- 20.1 to 20.6 map, unordered_map, set, container overview (PDF pages 591 to 609)

Algorithms

Over a hundred generic functions in `<algorithm>` / `<numeric>`. Prefer them to loops: named, tested, often faster. Most take a lambda.

Try some on `{5, 2, 9, 2, 7, 4}`:

PLAYGROUND **STL algorithms on {5, 2, 9, 2, 7, 4}**

Algorithm: sort ascending

```
std::sort(v.begin(), v.end());
```

Before: `{5, 2, 9, 2, 7, 4}`
After: `{2, 2, 4, 5, 7, 9}`

```
std::sort(v.begin(), v.end(), [](const P& a, const P& b) { return a.age < b.age; });   // custom order
auto it = std::find_if(v.begin(), v.end(), [](int x) { return x > 6; });
int n  = std::count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
bool any = std::any_of(v.begin(), v.end(), [](int x) { return x < 0; });
int sum  = std::accumulate(v.begin(), v.end(), 0);          // <numeric>
std::transform(v.begin(), v.end(), v.begin(), [](int x) { return x * x; });
auto pos = std::lower_bound(sorted.begin(), sorted.end(), 5);   // binary search: range MUST be sorted
```

▼ PREDICT The remove surprise

```
std::vector<int> v{1, 2, 3, 4, 5};
std::remove(v.begin(), v.end(), 3);
std::cout << v.size();
```

```
5
```

Algorithms cannot change a container's size. `std::remove` shifts kept elements forward and returns the new logical end. Use the **erase-remove idiom**:

```
v.erase(std::remove(v.begin(), v.end(), 3), v.end());   // classic
std::erase(v, 3);                                       // C++20: one call
```

STRICT WEAK ORDERING Use `<`, never `≤`: a comparator true for equal elements can crash `sort` or corrupt a `set`. `sort` is not stable; use `stable_sort`.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 10.1 to 10.2 Overview and a first look (PDF pages 405 to 413)

Programming: Principles and Practice (Stroustrup)

- 21.1 to 21.2, 21.5 Standard algorithms, function objects, sorting and searching (PDF pages 617 to 626 and 633 to 635)

C++ Move Semantics (Josuttis)

- Chapter 14 Moving algorithms and iterators (PDF pages 219 to 228)

Ranges and views (C++20)

```cpp
#include <ranges>

std::ranges::sort(v);                                   // pass the
container, not begin() and end()
std::ranges::sort(v, std::greater<>{});                 // descending
(compiled: first element 6)

auto evenSquares = v | std::views::filter([](int x) { return x % 2 == 0; })
                     | std::views::transform([](int x) { return x * x; });
// a LAZY view: nothing is computed yet
for (int x : evenSquares) use(x);                       // computed on
demand, no temporary vector (compiled: sum 56)
```

- A **range** is anything with `begin()` and `end()`. The `std::ranges::` algorithms take the whole range and support **projections**: `std::ranges::sort(people, {}, &Person::age)`.
- A **view** is a lightweight, non-owning, lazy window (`filter`, `transform`, `take`, `drop`, `reverse`, `iota`). Pipe them with `|`.
- A view does not own its data: if the container dies or reallocates, the view dangles, like `string_view`.
- Ranges make pipelines readable; you do not need to know every view.

Common pitfalls

`pair`, `tuple`, `optional` and `variant` are covered in Vocabulary Types.

PITFALL	FIX
Using an iterator/pointer/reference after a `push_back` reallocated the vector	`reserve` up front, or re-fetch the iterator, or index instead
`map[key]` silently inserts	use `find` / `contains` to test; `at()` to read-or-throw
`v.size() - 1` on an empty vector (`size()` is unsigned, so it wraps to a huge number)	check `empty()` first or write `i + 1 < v.size()`

Passing a big container by value	pass `const std::vector<T>&`
`std::vector<bool>` is a packed-bit special case with proxy references	use `std::vector<char>` or `std::bitset`
`auto x = v[0]` copies the element	`const auto& x = v[0]`
`std::find` on a `map` is O(n)	use the member `m.find(key)`, O(log n)

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 9.3 Sequential container operations (PDF pages 370 to 383)

Random numbers: `<random>`

```
std::mt19937 gen(12345);                          // an ENGINE: produces raw random bits (here seeded for repeatability)
std::uniform_int_distribution<int> dice(1, 6);    // a DISTRIBUTION: shapes the bits into the numbers you want
int roll = dice(gen);

std::random_device rd;                            // a non-deterministic seed source (hardware or OS)
std::mt19937 gen2(rd());
std::normal_distribution<double> noise(0.0, 1.0);
```

- **Engine plus distribution**: never use `rand() % n` (poor quality, biased, global state). Same seed gives the **same sequence** (compiled: true), which is what you want in tests and simulations.
- Compiled: 6000 rolls of the dice gave every face between 850 and 1150 times; 10,000 normal samples had a mean near 0.
- Keep one engine per thread; engines are not thread-safe.

Notes: Module 3.

READ MORE

C++ Primer (Lippman, Lajoie, Moo)

- 17.4 Random numbers (PDF pages 774 to 780)

Cheat sheet

<details>
<summary>▼ How do you use your own type as a key in <code>unordered_map</code>, and in <code>map</code>?</summary>

<code>unordered_map</code>: provide <code>operator==</code> and a hash functor (or specialise <code>std::hash</code>) that combines all members; equal objects must hash equally. <code>map</code>: provide a strict weak ordering (<code>operator<</code> or a comparator type). Never <code>≤</code>, and never modify a key while it is stored.
</details>

<details>
<summary>▼ Why use <code>try_emplace</code> instead of <code>emplace</code> or <code>insert</code>?</summary>

If the key exists, <code>try_emplace</code> does nothing and does not touch its arguments, so a moved-in value is not lost. <code>emplace</code> and <code>insert</code> may already have built or moved from the value. It is the right call for "insert if absent" with expensive or move-only values.
</details>

<details>
<summary>▼ What does <code>std::back_inserter</code> do and when do you need it?</summary>

It is an output iterator whose assignment calls <code>push_back</code>. Algorithms such as <code>copy</code> and <code>transform</code> write through iterators and do not grow the destination, so with an empty target you would write past the end; <code>back_inserter</code> makes them grow it instead.
</details>

▼ What are C++20 ranges and views?

Ranges algorithms take a whole container (`std::ranges::sort(v)`) and support projections. Views are lazy, non-owning pipelines (`v | views::filter(f) | views::transform(g)`) that compute on demand without temporaries. Like `string_view`, a view dangles if its container dies.

▼ How does `std::vector` grow?

When `size == capacity` it allocates a larger buffer (geometric growth, typically 1.5x to 2x, implementation-defined), moves or copies the elements, and frees the old buffer. This makes `push_back` amortised O(1), but it invalidates all iterators, pointers and references. `reserve` avoids the reallocations.

▼ `map` vs `unordered_map`?

`map`: balanced tree, sorted iteration, O(log n), needs `operator<`, supports range queries. `unordered_map`: hash table, average O(1) (worst O(n) with many collisions), no order, needs a hash and `==`. Choose `unordered_map` for plain lookups and `map` when you need order.

▼ `push_back` vs `emplace_back`?

`push_back` takes an existing object (a copy or move). `emplace_back` forwards its arguments to the element's constructor and builds it in place, avoiding a temporary. For simple types there is no difference; for objects with expensive construction `emplace_back` can save a move.

▼ What is the erase-remove idiom?

`std::remove` / `remove_if` only moves the elements to keep to the front and returns the new end; it cannot shrink the container. Pass that to the container's `erase` to actually delete the tail: `v.erase(std::remove_if( ... ), v.end())`. C++20 has `std::erase_if`.

▼ What is a hash collision and how is it handled?

Two different keys hash to the same bucket. The standard library's unordered containers resolve it with chaining (a bucket holds a list of entries), so lookups in a heavily colliding bucket degrade toward O(n). A good hash function and a sensible load factor (`rehash`, `reserve`) keep it near O(1).

▼ Why can't I `std::sort` a `std::list`?

`sort` needs random-access iterators (it jumps around by index). A list only provides bidirectional iterators, so it has its own member function `list::sort`.

▼ Which container for a task queue? For "top K" elements?

A queue (`std::queue`, backed by `deque`) for FIFO tasks. A `priority_queue` (heap) for top K: keep a heap of size K and compare against its root: O(n log K).